Apply filters to SQL queries

Project description

My focus is keeping our organization's systems secure. Day to day, that means monitoring for vulnerabilities, investigating potential threats, and making sure everyone's computer is fully updated. Below are a few examples of how I use SQL filters to handle these security tasks efficiently.

Retrieve after hours failed login attempts

We recently had a potential security incident that happened after hours. To look into it, I needed to check all failed login attempts that occurred after 18:00.

Here is the SQL query I used to filter the logs for those specific after-hours failures.

```
MariaDB [organization]> SELECT *
    -> FROM log_in_attempts
    -> WHERE login_time > '18:00' AND success = FALSE;
+----------+----------+------------+------------+---------+-----------------+---------+
| event_id | username | login_date | login_time | country | ip_address      | success |
+----------+----------+------------+------------+---------+-----------------+---------+
|        2 | apatel   | 2022-05-10 | 20:27:27   | CAN     | 192.168.205.12  |       0 |
|       18 | pwashing | 2022-05-11 | 19:28:50   | US      | 192.168.66.142  |       0 |
|       20 | tshah    | 2022-05-12 | 18:56:36   | MEXICO  | 192.168.109.50  |       0 |
```

The first part of the screenshot is my query, and the second part is a table of the output. To find the after hours issues, I pulled all data from the `log_in_attempts` table and used a `WHERE` clause with an `AND` operator to isolate the specific events. The query checks for two things: `login_time > '18:00'` to capture anything after business hours, and `success = FALSE` which filters for the failed attempts.

Retrieve login attempts on specific dates

Following a suspicious event on May 9, 2022, I needed to review all login activity from that day as well as the day before.

The SQL query below shows how I filtered the logs to pull data for those specific dates.

```
MariaDB [organization]> SELECT *
    -> FROM log_in_attempts
    -> WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
+----------+----------+------------+------------+---------+-----------------+---------+
| event_id | username | login_date | login_time | country | ip_address      | success |
+----------+----------+------------+------------+---------+-----------------+---------+
|        1 | jrafael  | 2022-05-09 | 04:56:27   | CAN     | 192.168.243.140 |       0 |
|        3 | dkot     | 2022-05-09 | 06:47:41   | USA     | 192.168.151.162 |       0 |
|        4 | dkot     | 2022-05-08 | 02:00:39   | USA     | 192.168.178.71  |       0 |
```

The screenshot shows my query along with a sample of the results. To pull the data, I selected everything from the `log_in_attempts` table and used a `WHERE` clause with an `OR` operator. This allowed me to target both dates at once, checking if `login_date` equaled either `'2022-05-09'` or `'2022-05-08'`.

Retrieve login attempts outside of Mexico

While reviewing the login logs, I found a red flag with the attempts originating outside of Mexico. These look risky and need a proper look.

I wrote this SQL query to isolate those international logins:

```
MariaDB [organization]> SELECT *
    -> FROM log_in_attempts
    -> WHERE NOT country LIKE 'MEX%';
+----------+----------+------------+------------+---------+-----------------+---------+
| event_id | username | login_date | login_time | country | ip_address      | success |
+----------+----------+------------+------------+---------+-----------------+---------+
|        1 | jrafael  | 2022-05-09 | 04:56:27   | CAN     | 192.168.243.140 |       0 |
|        2 | apatel   | 2022-05-10 | 20:27:27   | CAN     | 192.168.205.12  |       0 |
|        3 | dkot     | 2022-05-09 | 06:47:41   | USA     | 192.168.151.162 |       0 |
```

The top half of the screenshot shows my query, and the bottom shows a snippet of the results. I started by pulling everything from the `log_in_attempts` table, then used `WHERE` `NOT` to filter out any logins from Mexico. Because the dataset labels Mexico as both 'MEX' and 'MEXICO,' I used `LIKE` `MEX%` the percentage sign (%) acts as a wildcard to catch both variations.

Retrieve employees in Marketing

To facilitate hardware upgrades for specific Marketing department personnel, I needed to identify the target machines. The SQL query below demonstrates how I filtered for devices assigned to Marketing employees located in the East building.

```
MariaDB [organization]> SELECT *
    -> FROM employees
    -> WHERE department = 'Marketing' AND office LIKE 'East%';
+-------------+--------------+----------+------------+----------+
| employee_id | device_id    | username | department | office   |
+-------------+--------------+----------+------------+----------+
|        1000 | a320b137c219 | elarson  | Marketing  | East-170 |
|        1052 | a192b174c940 | jdarosa  | Marketing  | East-195 |
|        1075 | x573y883z772 | fbautist | Marketing  | East-267 |
```

The screenshot displays my query followed by a sample of its output, which lists all Marketing department employees located in the East building.

To achieve this, I selected all records from the `employees` table and applied a `WHERE` clause with an `AND` operator to combine two conditions. The first condition (`department = 'Marketing'`) isolates the correct department. The second condition (`office LIKE 'East%'`) uses a wildcard to capture all employees in the East building, account for specific office numbers stored in that column.

Retrieve employees in Finance or Sales

Employees in the Finance and Sales departments require a different security update, meaning their machine data must be pulled separately. The SQL query below demonstrates how I isolated records for personnel belonging to either of these two departments.

```
MariaDB [organization]> SELECT *
    -> FROM employees
    -> WHERE department = 'Finance' OR department = 'Sales';
+-------------+--------------+----------+------------+-------------+
| employee_id | device_id    | username | department | office      |
+-------------+--------------+----------+------------+-------------+
|        1003 | d394e816f943 | sgilmore | Finance    | South-153   |
|        1007 | h174i497j413 | wjaffrey | Finance    | North-406   |
|        1008 | i858j583k571 | abernard | Finance    | South-170   |
```

The screenshot displays the SQL query alongside a sample of the resulting output, which includes all employees from both the Finance and Sales departments.

After selecting all data from the `employees` table, I applied a `WHERE` clause using the `OR` operator. Because an employee cannot belong to both departments simultaneously, `OR`

ensures the query retrieves anyone who matches either criteria. The first condition (`department = 'Finance'`) pulls Finance department, while the second condition (`department = 'Sales'`) includes the Sales team.

Retrieve all employees not in IT

The final security update targets all personnel outside of the Information Technology department. To gather the necessary machine data, I used the following SQL query to exclude IT department employees from the results.

```
MariaDB [organization]> SELECT *
    -> FROM employees
    -> WHERE NOT department = 'Information Technology';
+-------------+--------------+----------+-----------------+-------------+
| employee_id | device_id    | username | department      | office      |
+-------------+--------------+----------+-----------------+-------------+
|        1000 | a320b137c219 | elarson  | Marketing       | East-170    |
|        1001 | b239c825d303 | bmoreno  | Marketing       | Central-276 |
|        1002 | c116d593e558 | tshah    | Human Resources | North-434   |
```

The upper half of the screenshot is my query, and the lower half is a table of the output. The query returned all employees not in the Information Technology department. First, I started by selecting all data from the `employees` table. Then, I used a `WHERE` clause with `NOT` to filter for employees not in this department.

Summary

Filtered SQL queries across the `log_in_attempts` and `employees` tables to extract specific data on user logins and device assignments.
Utilized logical operators (`AND`, `OR`, `NOT`) and string matching (`LIKE` with `%` wildcard) to isolate targeted data patterns for various security and administrative tasks.